DL4

Recurrent neural network (RNN) Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.

```
import numpy as np                  # For numerical operations
import pandas as pd                 # For data handling (CSV, tables)
import matplotlib.pyplot as plt     # For plotting graphs

from sklearn.preprocessing import MinMaxScaler   # For scaling data between 0 and 1
from tensorflow.keras.models import Sequential   # To build neural network model
from tensorflow.keras.layers import Dense, SimpleRNN  # Layers used in RNN

# 2. Load Dataset
df = pd.read_csv("Google_Stock_Price.csv", thousands=',')
# Read CSV file, thousands=',' handles values like 1,000 properly

# Take only 'Open' column (stock opening price)
data = df['Open'].values.reshape(-1, 1)

# Create scaler object (normalize data between 0 and 1)
scaler = MinMaxScaler(feature_range=(0,1))

# Convert data to numeric (remove invalid values like text), drop NaN values
data = pd.to_numeric(df['Open'], errors='coerce').dropna().values.reshape(-1, 1)

# Scale the data
data_scaled = scaler.fit_transform(data)
```

```
# Split dataset into training (80%) and testing (20%)
train_size = int(len(data_scaled) * 0.8)

train_data = data_scaled[:train_size]
test_data = data_scaled[train_size:]

# Function to create dataset with time steps (60 previous days → next day prediction)
def create_dataset(dataset):
    X = []   # Input sequences
    y = []   # Output values

    # Loop starts from 60 because we need past 60 days data
    for i in range(60, len(dataset)):
        X.append(dataset[i-60:i, 0])  # Last 60 values as input
        y.append(dataset[i, 0])       # Current value as output

    return np.array(X), np.array(y)

# Create training and testing datasets
X_train, y_train = create_dataset(train_data)
X_test, y_test = create_dataset(test_data)

# Reshape input to 3D (samples, time steps, features)
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

# Build RNN model
model = Sequential()
```

```
# First RNN layer with 50 neurons, returns sequences for next layer
model.add(SimpleRNN(50, return_sequences=True, input_shape=(60,1)))

# Second RNN layer with 50 neurons
model.add(SimpleRNN(50))

# Output layer (1 neuron → predicted price)
model.add(Dense(1))

# Compile model with Adam optimizer and MSE loss
model.compile(optimizer='adam', loss='mean_squared_error')

# Show model structure
model.summary()

# Train the model (20 epochs, batch size = 32)
model.fit(X_train, y_train, epochs=20, batch_size=32)

# Predict on test data
predicted = model.predict(X_test)

# Convert scaled values back to original price
predicted = scaler.inverse_transform(predicted)
real = scaler.inverse_transform(y_test.reshape(-1,1))

# Plot real vs predicted prices
plt.plot(real, color='red', label='Real Price')
plt.plot(predicted, color='blue', label='Predicted Price')
```

```
plt.title("Google Stock Price Prediction (RNN)")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```

🔹 1. Importing Libraries

import numpy as np

👉 NumPy is used for numerical operations and handling arrays.

import pandas as pd

👉 Pandas is used to read and manage dataset (CSV file).

import matplotlib.pyplot as plt

👉 Used to plot graphs (visualize stock prices).

from sklearn.preprocessing import MinMaxScaler

👉 This is used to **normalize data between 0 and 1**, which helps the neural network learn better.

from tensorflow.keras.models import Sequential

👉 Sequential is used to create a neural network layer by layer.

from tensorflow.keras.layers import Dense, SimpleRNN

👉 Dense = fully connected layer
👉 SimpleRNN = Recurrent Neural Network layer (used for time series data)

🔹 2. Loading Dataset

df = pd.read_csv("Google_Stock_Price.csv", thousands=',')

👉 Reads the CSV file.
👉 thousands=',' handles values like 1,000 correctly.

🔹 3. Selecting Data

data = df['Open'].values.reshape(-1, 1)

👉 We are selecting only the **Open price column**.
👉 .reshape(-1,1) converts it into 2D array (required format for ML).

🔹 4. Scaling Data

scaler = MinMaxScaler(feature_range=(0,1))

👉 Creates scaler object to convert values between 0 and 1.

```
data = pd.to_numeric(df['Open'], errors='coerce').dropna().values.reshape(-1, 1)
```

👉 Converts values into numeric format.
👉 Removes invalid values (like text or missing values).

```
data_scaled = scaler.fit_transform(data)
```

👉 Scales the data between 0 and 1.

🔹 5. Splitting Dataset

```
train_size = int(len(data_scaled) * 0.8)
```

👉 80% data is used for training.

```
train_data = data_scaled[:train_size]
test_data = data_scaled[train_size:]
```

👉 Remaining 20% is used for testing.

🔹 6. Creating Time-Series Dataset

```
def create_dataset(dataset):
```

👉 This function prepares data for RNN.

```
    X = []
    y = []
```

👉 X = input (past values)
👉 y = output (next value)

```
    for i in range(60, len(dataset)):
```

👉 We take **last 60 days data** to predict next day.

```
        X.append(dataset[i-60:i, 0])
```

👉 Store last 60 values as input.

```
        y.append(dataset[i, 0])
```

👉 Store current value as output.

```
    return np.array(X), np.array(y)
```

👉 Convert lists into arrays.

🔹 7. Prepare Training & Testing Data

```
X_train, y_train = create_dataset(train_data)
X_test, y_test = create_dataset(test_data)
```

👉 Generate input-output pairs for training and testing.

🔹 8. Reshaping Data for RNN

```
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
```

👉 RNN expects 3D input:

- samples
- time steps (60 days)
- features (1 value per day)

🔹 9. Building RNN Model

```
model = Sequential()
```

👉 Initialize neural network.

```
model.add(SimpleRNN(50, return_sequences=True, input_shape=(60,1)))
```

👉 First RNN layer with 50 neurons.
👉 return_sequences=True → passes output to next RNN layer.

```
model.add(SimpleRNN(50))
```

👉 Second RNN layer.

```
model.add(Dense(1))
```

👉 Output layer → predicts 1 value (stock price).

🔹 10. Compile Model

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

👉 Adam optimizer → updates weights
👉 MSE → calculates error between actual and predicted values

🔹 11. Model Summary

model.summary()

👉 Shows structure of neural network.

🔹 12. Training Model

model.fit(X_train, y_train, epochs=20, batch_size=32)

👉 Train model:

- epochs = 20 (iterations)
- batch_size = 32 (data processed in batches)

🔹 13. Prediction

predicted = model.predict(X_test)

👉 Predict stock prices using test data.

🔹 14. Convert Back to Original Values

predicted = scaler.inverse_transform(predicted)

👉 Convert predicted values back to original price scale.

real = scaler.inverse_transform(y_test.reshape(-1,1))

👉 Convert actual values back.

🔹 15. Plot Graph

plt.plot(real, color='red', label='Real Price')

👉 Plot actual stock prices.

plt.plot(predicted, color='blue', label='Predicted Price')

👉 Plot predicted prices.

plt.title("Google Stock Price Prediction (RNN)")
plt.xlabel("Time")

```
plt.ylabel("Price")
plt.legend()
plt.show()
```

👉 Display final graph comparison.

🔥 **Final Viva Summary (Say This)**

👉 *"This project uses a Simple RNN model to predict Google stock prices based on past 60 days data. The dataset is normalized, split into training and testing sets, and the model is trained using Adam optimizer with MSE loss. Finally, predicted values are compared with actual values using a graph."*

🔥 **Dry Run of Your Code (Simple Example)**

To understand clearly, let's take a **small sample dataset** instead of full Google data:

👉 Assume Open prices:

[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

🔹 **Step 1: Scaling**

After MinMaxScaler (0 to 1):

10 → 0.0
20 → 1.0

Scaled data becomes:

[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]

🔹 **Step 2: Train-Test Split (80%)**

Train: [0.0 → 0.8]
Test : [0.9, 1.0]

🔹 **Step 3: create_dataset() Logic**

👉 In real code → uses **60 previous days**
👉 For dry run → assume **window = 3** (for simplicity)

Training Data Example:

Train = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]

Loop starts from index 3:

Iteration 1:

X = [0.0, 0.1, 0.2]
y = 0.3

Iteration 2:

X = [0.1, 0.2, 0.3]
y = 0.4

Iteration 3:

X = [0.2, 0.3, 0.4]
y = 0.5

Final Output:

```
X_train =
[
 [0.0, 0.1, 0.2],
 [0.1, 0.2, 0.3],
 [0.2, 0.3, 0.4]
]

y_train =
[0.3, 0.4, 0.5]
```

🔹 Step 4: Reshaping

Before reshape:

X_train shape = (3, 3)

After reshape:

X_train shape = (3, 3, 1)

👉 Meaning:

- 3 samples
- 3 time steps
- 1 feature

🔹 Step 5: Model Training

Model learns pattern like:

👉 "If past values are increasing → next value will increase"

Internally:

- weights are adjusted
- error is minimized using MSE

🔹 Step 6: Prediction

Test data:

[0.9, 1.0]

Using previous values, model predicts next:

Predicted ≈ 1.1 (scaled value)

🔹 Step 7: Inverse Scaling

Convert back to original scale:

1.1 → approx 21

🔹 Step 8: Graph Output

Final graph shows:

- 🔴 Red line → Actual price
- 🔵 Blue line → Predicted price

👉 If both lines are close → model is good

🎯 Final Viva Explanation (Say This)

👉 *"In dry run, the model takes past values (like 60 days in real case), forms input-output pairs, trains on those sequences, and learns patterns. Then it predicts future values, which are converted back to original scale and compared with actual values using a graph."*

⚠️ Important Point (Exam Trick)

If examiner asks:

👉 **Why 60 days?**
✓ "To capture time dependency and trend in stock prices"

👉 **Why scaling?**
✓ "Neural networks perform better when data is normalized"

🔥 **1. What is this project about?**

👉 **Answer:**
This project predicts stock prices using a **Recurrent Neural Network (RNN)**. It uses past 60 days' stock prices to predict the next day's price.

🔥 **2. Why did you use RNN?**

👉 **Answer:**
RNN is suitable for **time-series data** because it remembers past information and uses it to predict future values.

🔥 **3. What is time-series data?**

👉 **Answer:**
Time-series data is data collected over time in sequence, like stock prices, weather data, etc.

🔥 **4. Why did you choose 60 days?**

👉 **Answer:**
60 days are used as a **window size** to capture trends and patterns. It helps the model understand short-term dependencies.

🔥 **5. What is MinMaxScaler?**

👉 **Answer:**
MinMaxScaler scales data between **0 and 1**, which helps neural networks train faster and more accurately.

🔥 **6. Why is scaling important?**

👉 **Answer:**
Without scaling, large values can dominate smaller ones, and training becomes unstable.

🔥 **7. What is the role of create_dataset()?**

👉 **Answer:**
It converts raw data into **input-output sequences**, where past 60 values are inputs and the next value is output.

🔥 **8. Why do we reshape data into 3D?**

👉 **Answer:**
RNN requires input in 3D format:
👉 (samples, time steps, features)

🔥 **9. What is SimpleRNN?**

👉 **Answer:**
SimpleRNN is a basic recurrent neural network layer that processes sequences and maintains memory of previous inputs.

🔥 **10. What is return_sequences=True?**

👉 **Answer:**
It returns the full sequence output so it can be passed to the next RNN layer.

🔥 **11. What is Dense layer?**

👉 **Answer:**
Dense is a fully connected layer that produces the final output (predicted value).

🔥 **12. What optimizer did you use and why?**

👉 **Answer:**
Adam optimizer is used because it is efficient and adapts learning rate automatically.

🔥 **13. What is Mean Squared Error (MSE)?**

👉 **Answer:**
MSE calculates the average squared difference between actual and predicted values.

🔥 **14. What is an epoch?**

👉 **Answer:**
One complete pass of the entire training dataset through the model.

🔥 **15. What is batch size?**

👉 **Answer:**
Number of samples processed before updating model weights.

🔥 **16. What happens during training?**

👉 **Answer:**
The model learns patterns by adjusting weights to minimize prediction error.

🔥 **17. Why do we use inverse_transform?**

👉 **Answer:**
To convert scaled values back to original stock prices for meaningful interpretation.

🔥 **18. How do you evaluate the model?**

👉 **Answer:**
By comparing predicted vs actual values using graphs and error metrics.

🔥 **19. What are limitations of SimpleRNN?**

👉 **Answer:**

- Cannot handle long-term dependencies well
- Suffers from vanishing gradient problem

🔥 **20. How can you improve this model?**

👉 **Answer:**

- Use LSTM or GRU
- Increase dataset size
- Tune hyperparameters

- Add more features (volume, close price)

🔥 **21. Difference between RNN and LSTM?**

👉 **Answer:**
LSTM is an improved version of RNN that can remember long-term dependencies using gates.

🔥 **22. What is overfitting?**

👉 **Answer:**
When model performs well on training data but poorly on new data.

🔥 **23. How to avoid overfitting?**

👉 **Answer:**

- Use more data
- Add dropout
- Reduce model complexity

🔥 **24. Why use Sequential model?**

👉 **Answer:**
Because layers are added in a simple linear stack.

🔥 **25. What is feature in your model?**

👉 **Answer:**
Only one feature → stock "Open price".

🔥 **26. What is output of your model?**

👉 **Answer:**
Predicted stock price for the next day.

🔥 **27. Why only 'Open' price used?**

👉 **Answer:**
For simplicity. More features can improve accuracy.

🔥 **28. Can this model be used in real trading?**

👉 **Answer (smart answer):**
It gives prediction trends but real trading requires more factors like news, market conditions, and risk analysis.

🔥 **29. What is vanishing gradient problem?**

👉 **Answer:**
Gradients become very small during training, making learning slow or ineffective.

🔥 **30. What is deep learning?**

👉 **Answer:**
A subset of machine learning that uses neural networks with multiple layers to learn complex patterns.

🎯 **Final Smart Closing Answer (Impress Examiner)**

👉 *"This model demonstrates how sequential data can be used for prediction using RNN, but for better accuracy and real-world application, advanced models like LSTM and additional features should be used."*